

---

# Analisi Tecnica del Firmware

Progetto *Vera-Module-RBL-XE*

---

Analisi del progetto a cura di RetroBit Lab

10 giugno 2026

# Indice

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                        | <b>1</b>  |
| <b>2</b> | <b>Architettura del Sistema</b>                            | <b>1</b>  |
| 2.1      | Componenti Chiave . . . . .                                | 1         |
| 2.2      | Modalità operative (VERA_BOARD_IS_PBI) . . . . .           | 1         |
| 2.3      | Ambiente di build PlatformIO . . . . .                     | 2         |
| 2.4      | Interfacciamento Bus . . . . .                             | 2         |
| <b>3</b> | <b>Mapping dei Pin (ESP32-S3FN8, QFN56)</b>                | <b>3</b>  |
| 3.1      | Quarzo oscillatore 40 MHz (pin 55–56) . . . . .            | 5         |
| <b>4</b> | <b>Analisi delle Tempistiche</b>                           | <b>6</b>  |
| 4.1      | Accesso diretto ai registri GPIO . . . . .                 | 6         |
| 4.2      | Diagramma Temporale di un Ciclo di Lettura (PBI) . . . . . | 6         |
| <b>5</b> | <b>Frammento di Codice Sorgente</b>                        | <b>7</b>  |
| <b>6</b> | <b>Diagramma di Flusso del Firmware</b>                    | <b>9</b>  |
| <b>7</b> | <b>Conclusione</b>                                         | <b>11</b> |

# 1 Introduzione

Questo documento presenta un'analisi tecnica del firmware per il progetto *Vera-Module-RBL-XE*. L'obiettivo del firmware, eseguito su un microcontrollore ESP32, è di interfacciarsi con il bus di un processore 6502 per emulare un dispositivo PBI (Parallel Bus Interface) o CCTL (Cartridge Control Line) per computer ATARI della serie XL/XE.

Il progetto supporta la MCU target ESP32-S3 e due modalità operative, per un totale di due varianti di compilazione distinte. L'analisi si concentra sulla fattibilità temporale (timing) dell'interazione tra la MCU moderna e la CPU vintage, illustrando la logica di funzionamento tramite diagrammi di flusso e temporali.

## 2 Architettura del Sistema

Il sistema si basa sull'interazione di due componenti principali.

### 2.1 Componenti Chiave

- **CPU 6502 (MOS/SALLY):** Il processore centrale dell'Atari, operante a una frequenza di clock di **1.7897 MHz** (NTSC).
- **MCU:** Il microcontrollore moderno responsabile del monitoraggio del bus e dell'emulazione della periferica. Il target è:
  - **ESP32-S3FN8** (QFN56, 7×7 mm) — Dual-core Xtensa LX7 @ 240 MHz, 512 KB SRAM, 8 MB flash in-package Quad SPI, 45 GPIO bidirezionali.

### 2.2 Modalità operative (VERA\_BOARD\_IS\_PBI)

Il firmware supporta due modalità operative, selezionate a compile-time tramite la macro `VERA_BOARD_IS_PBI` in `main.cpp`:

- **PBI mode (VERA\_BOARD\_IS\_PBI=1, default):** Il modulo si collega al connettore ECI della serie XL/XE.
  - **VCS latch:** scrittura di \$80 a \$D1FF attiva il dispositivo; qualsiasi altro valore lo disattiva.
  - **ROM \$D800–\$DFFF (2 KB, 11 bit A0–A10):** MPD (Math Pack Disable, GPIO42) viene asserito basso per ogni ciclo ROM con device selezionato, disabilitando la ROM Math Pack interna all'Atari.
  - **RAM \$D600–\$D7FF (512 B, 9 bit A0–A8):** buffer IRAM esposto al 6502. `EXTSEL_N` viene asserito per-ciclo.
  - **Registri VERA \$D100–\$D11F (32 registri):** `DEV_SEL_N` e `EXTSEL_N` affermati quando selezionato; la FPGA VERA guida il bus dati.
  - **Intervallo \$D120–\$D1FE:** `EXTSEL_N` asserito; nessun dato guidato dall'ESP32 (non implementato nel firmware corrente).

- **EXTSEL\_N (GPIO41)**: segnale *per-ciclo* che disabilita MMU/Freddie dell'Atari per accessi D1xx e D6xx. Non è un segnale persistente.
- **CCTL mode (VERA\_BOARD\_IS\_PBI=0)**: Il modulo si collega alla porta cartuccia. Il firmware decodifica l'indirizzo completo a 16 bit e intercetta \$D500–\$D5FF: DEV\_SEL\_N affermato per \$D500–\$D5FE (FPGA guida il bus dati), latch di selezione a \$D5FF (scrittura di \$80 seleziona la scheda).

## 2.3 Ambiente di build PlatformIO

Il file `platformio.ini` definisce un unico ambiente unificato:

| Ambiente   | MCU         | Modalità                        | Piattaforma       |
|------------|-------------|---------------------------------|-------------------|
| esp32s3fn8 | ESP32-S3FN8 | PBI + CCTL (unificato, runtime) | espressif32@6.6.0 |

La modalità PBI/CCTL non è più selezionata da ambienti separati ma dalla macro `VERA_BOARD_IS_PBI` in `main.cpp`: valore 1 per PBI (default), 0 per CCTL.

## 2.4 Interfacciamento Bus

L'ESP32 legge il bus indirizzi completo **A0–A15** e i segnali **PHI2** e **R/W\_** direttamente dai propri GPIO ad ogni ciclo di clock. **Non esiste alcuna logica esterna di pre-decodifica degli indirizzi**: i segnali **D1XX\_N**, **ROM\_SEL\_N** e **RAM\_SEL\_N** del progetto precedente sono stati eliminati. La decodifica è interamente software in `decode_addr()`, che restituisce un indirizzo a 16 bit senza ambiguità di pagina.

I quattro TXS0108E (U1–U4) svolgono esclusivamente adattamento di livello 5 V ↔ 3.3 V:

- **U1 — Bus Dati D0–D7 (GPIO 4–11, Bank-0 bit 4–11)**: bidirezionale. Il firmware abilita le uscite con `GPIO.enable_w1ts` solo durante i cicli di lettura che richiedono risposta dell'ESP32 (ROM \$D800–\$DFFF, RAM \$D600–\$D7FF, RAMbo \$4000–\$7FFF).
- **U2 — Bus Indirizzi A0–A7 (GPIO 12–19, Bank-0 bit 12–19)**: tutti ingressi B→A (Atari 5 V → ESP32 3.3 V). GPIO 19 condivide il pad USB D-; USB CDC è disabilitato nel firmware.
- **U3 — A8–A11, PHI2, R/W\_, EXTSEL\_N, MPD (GPIO 20–21, 35–36, 1, 2, 41, 42)**: ingressi B→A per A8, A9, A10, A11, PHI2, R/W\_; uscite A→B per EXTSEL\_N (GPIO 41) e MPD (GPIO 42). **ARESET** (GPIO 37) è anch'esso su U3 (uscita A→B). I GPIO 39–42 erano riservati a JTAG: `setup()` li de-registra con `gpio_reset_pin()` prima di creare il Dedicated GPIO bundle.
- **U4 — Bus Indirizzi A12–A15 (GPIO 33–34, 47–48, Bank-1)**: tutti ingressi B→A (4 canali su 8 utilizzati). GPIO 33 e GPIO 34 si trovano sul lato opposto del QFN56 rispetto a GPIO 47 e GPIO 48: applicare *length matching* (serpentine alla traccia più lunga) per eliminare lo skew su U4.

- **Connessioni dirette (senza level shifter, 3.3 V):** DEV\_SEL\_N (GPIO 40) → VERA /CS; CRESET (GPIO 38) → VERA /RESET; CDONE (GPIO 39) ← VERA CDONE.

I segnali DEV\_SEL\_N, EXTSEL\_N e MPD sono raggruppati nel **Dedicated GPIO output bundle** (Core 1, canali 0–2): vengono asseriti e de-asseriti in un'unica istruzione TIE Xtensa (`ee.wr_mask_gpio_out`,  $\approx 1$  ciclo CPU), anziché con tre scritture APB separate ( $\approx 18$  cicli totali). Il segnale PHI2 è nel bundle di ingresso (canale 0): il polling avviene via `ee.get_gpio_in` (1 ciclo CPU).

### 3 Mapping dei Pin (ESP32-S3FN8, QFN56)

I pin fisici del package QFN56 sono numerati 1–56 (più il pad GND centrale). La tabella seguente riporta i segnali del progetto con il numero GPIO logico e il numero di pad fisico del QFN56 (da Table 2-1 del datasheet ESP32-S3 v2.2).

**Nota importante:** il MCU *richiede* un **quarzo esterno a 40 MHz** collegato ai pin dedicati analogici **XTAL\_P** (pin 56) e **XTAL\_N** (pin 55). L'oscillatore RC interno è limitato al dominio RTC e non è in grado di pilotare il PLL della CPU a 240 MHz; senza quarzo il chip non si avvia. Vedere la sezione [3.1](#) per il circuito consigliato.

| Segnale                                                                 | GPIO | Pin QFN56 | IO MUX     | Dir. | Note                               |
|-------------------------------------------------------------------------|------|-----------|------------|------|------------------------------------|
| <i>Bus dati D0–D7 — bidirezionale via TXS0108E U1 (Bank-0 bit 4–11)</i> |      |           |            |      |                                    |
| D0                                                                      | 4    | 9         | GPIO4      | I/O  | glitch LOW 60 µs al power-up       |
| D1                                                                      | 5    | 10        | GPIO5      | I/O  |                                    |
| D2                                                                      | 6    | 11        | GPIO6      | I/O  |                                    |
| D3                                                                      | 7    | 12        | GPIO7      | I/O  |                                    |
| D4                                                                      | 8    | 13        | GPIO8      | I/O  |                                    |
| D5                                                                      | 9    | 14        | GPIO9      | I/O  |                                    |
| D6                                                                      | 10   | 15        | GPIO10     | I/O  |                                    |
| D7                                                                      | 11   | 16        | GPIO11     | I/O  |                                    |
| <i>Bus indirizzi A0–A7 — input via TXS0108E U2 (Bank-0 bit 12–19)</i>   |      |           |            |      |                                    |
| A0                                                                      | 12   | 17        | GPIO12     | IN   |                                    |
| A1                                                                      | 13   | 18        | GPIO13     | IN   |                                    |
| A2                                                                      | 14   | 19        | GPIO14     | IN   |                                    |
| A3                                                                      | 15   | 21        | XTAL_32K_P | IN   | pad RTC usato come GPIO; pin 20    |
| A4                                                                      | 16   | 22        | XTAL_32K_N | IN   | pad RTC usato come GPIO            |
| A5                                                                      | 17   | 23        | GPIO17     | IN   |                                    |
| A6                                                                      | 18   | 24        | GPIO18     | IN   |                                    |
| A7                                                                      | 19   | 25        | USB_D–     | IN   | USB CDC disabilitato               |
| <i>Bus indirizzi A8–A11 e controllo — via TXS0108E U3</i>               |      |           |            |      |                                    |
| A8                                                                      | 20   | 26        | USB_D+     | IN   | USB CDC disabilitato               |
| A9                                                                      | 21   | 27        | GPIO21     | IN   |                                    |
| A10                                                                     | 35   | 40        | GPIO35     | IN   | Bank-1 bit 3                       |
| A11                                                                     | 36   | 41        | GPIO36     | IN   | Bank-1 bit 4                       |
| PHI2                                                                    | 1    | 6         | GPIO1      | IN   | Dedicated GPIO input ch 0; 1.79 M  |
| R/W_                                                                    | 2    | 7         | GPIO2      | IN   | HIGH = lettura                     |
| EXTSEL_N                                                                | 41   | 46        | MTDI       | OUT  | Dedicated GPIO output ch 1; ex-J   |
| MPD                                                                     | 42   | 48        | MTMS       | OUT  | Dedicated GPIO output ch 2; ex-J   |
| ARESET                                                                  | 37   | 42        | GPIO37     | OUT  | Reset sistema Atari (LOW asserito) |
| <i>Bus indirizzi A12–A15 — input via TXS0108E U4 (Bank-1)</i>           |      |           |            |      |                                    |
| A12                                                                     | 33   | 38        | GPIO33     | IN   | Bank-1 bit 1; length matching con  |
| A13                                                                     | 34   | 39        | GPIO34     | IN   | Bank-1 bit 2                       |
| A14                                                                     | 47   | 53        | FSPICLK    | IN   | Bank-1 bit 15; FSPI non usato (fla |
| A15                                                                     | 48   | 54        | FSPID      | IN   | Bank-1 bit 16                      |
| <i>Segnali diretti (senza level shifter, 3.3 V)</i>                     |      |           |            |      |                                    |
| DEV_SEL_N                                                               | 40   | 45        | MTDO       | OUT  | VERA /CS; Dedicated GPIO outp      |
| CRESET                                                                  | 38   | 43        | GPIO38     | OUT  | VERA /RESET                        |
| CDONE                                                                   | 39   | 44        | MTCK       | IN   | VERA configurata (HIGH = OK);      |
| RAMBO_EN                                                                | 3    | 8         | GPIO3      | IN   | RAMbo enable: pull-up 10 kΩ→V      |
| <i>Debug</i>                                                            |      |           |            |      |                                    |
| UART TX                                                                 | 43   | 49        | U0TXD      | OUT  | Serial debug @ 115200 baud         |
| UART RX                                                                 | 44   | 50        | U0RXD      | IN   | non utilizzato in firmware         |

## Decodifica indirizzo (16 bit)

`decode_addr()` restituisce l'indirizzo completo A0–A15 (16 bit), catturato in due letture APB (`GPIO.in` per A0–A9 e `GPIO.in1.val` per A10–A15) già eseguite per il campionamento di PHI2. Le maschere di accesso ai buffer IRAM sono:

ROM \$D800–\$DFFF : `pbi_driver[addr & 0x7FF]` (2048 B, 11 bit A0–A10)

RAM \$D600–\$D7FF : `ram_pbi[addr & 0x1FF]` (512 B, 9 bit A0–A8)

RAMbo \$4000–\$7FFF : `extended_rambo_256k[(bank << 14) | (addr & 0x3FFF)]` (256 KB, 16 ban

## GPIO disponibili

Con A12–A15 aggiunti e `RAMBO_EN` (GPIO 3), tutti i GPIO utilizzabili sono ora assegnati. Non rimangono GPIO liberi per usi futuri (ad eccezione dei pin di strapping, che devono essere lasciati liberi).

**Attenzione:** I pin QFN56 28 e 30–35 (GPIO 26–32) sono collegati internamente alla flash Quad SPI in-package (variante FN8) e non devono mai essere connessi esternamente. GPIO 45 (pin 51) e GPIO 46 (pin 52) sono pin di strapping: lasciare liberi. I pin 55 (`XTAL_N`) e 56 (`XTAL_P`) sono pad analogici dedicati al quarzo 40 MHz: riservati, non usabili come GPIO (vedere sezione 3.1).

### 3.1 Quarzo oscillatore 40 MHz (pin 55–56)

L'ESP32-S3FN8 richiede un quarzo esterno da **40 MHz** per generare il riferimento PLL interno che alimenta la CPU a 240 MHz, l'interfaccia flash e i divisori UART. I pin sono *analogici dedicati*: non configurabili via software, non condivisi con GPIO.

| Pin QFN56 | Segnale | Note                                                                  |
|-----------|---------|-----------------------------------------------------------------------|
| 55        | XTAL_N  | Terminale negativo del cristallo (uscita amplificatore oscillatore)   |
| 56        | XTAL_P  | Terminale positivo del cristallo (ingresso amplificatore oscillatore) |

Schema consigliato:

```
XTAL_P (56) --[C1 10pF]-- GND
|
[X1 40MHz]    <- Rs 0-33 Ohm in serie (opzionale)
|
XTAL_N (55) --[C2 10pF]-- GND
```

| Componente                      | Valore                    | Note                                                              |
|---------------------------------|---------------------------|-------------------------------------------------------------------|
| X1 – quarzo                     | 40 MHz, SMD 3225 o 2016   | $\pm 10$ ppm; verificare $C_L$ dal datasheet                      |
| C1, C2 – condensatori di carico | 10 pF, 0402, $\pm 5\%$    | $C_{ext} = 2C_L - C_{stray}$ ; $C_{stray} \approx 3\text{--}5$ pF |
| Rs – resistenza serie           | 0–33 $\Omega$ (opzionale) | Limita il drive dell'oscillatore                                  |

**Regole di layout PCB:** tracce XTAL\_P/N più corte possibile (<5 mm); condensatori di carico posizionati ai pad del chip, non al corpo del quarzo; piano di massa solido sotto l'area del quarzo; isolamento dalle tracce PHI2, clock CPU 240 MHz e bus dati D0–D7.

## 4 Analisi delle Tempistiche

Una delle principali preoccupazioni in un progetto di questo tipo è la capacità della MCU di rispondere entro la finestra temporale definita dal ciclo di clock della CPU 6502.

- **Ciclo Clock 6502 (1.7897 MHz, NTSC):**  $T_{6502} = 1/(1.7897 \times 10^6) \approx 559 \text{ ns}$
- **Finestra PHI2 alto (half-cycle):**  $T_{PHI2\uparrow} \approx 279 \text{ ns}$  — questa è la finestra critica entro cui l'ESP32 deve leggere l'indirizzo, decodificarlo ed eventualmente pilotare il bus dati.
- **Ciclo Clock ESP32 (240 MHz):**  $T_{ESP32} = 1/(240 \times 10^6) \approx 4.17 \text{ ns}$

Il rapporto tra la finestra critica e il ciclo dell'ESP32 è:

$$\frac{T_{PHI2\uparrow}}{T_{ESP32}} = \frac{279 \text{ ns}}{4.17 \text{ ns}} \approx 67 \text{ cicli ESP32}$$

Anche considerando solo la semi-finestra di PHI2 alto (il caso peggiore), l'ESP32-S3 dispone di circa **67 cicli di clock** per completare tutta la catena: lettura di `GPIO.in` / `GPIO.in1.val`, decodifica dell'indirizzo, lookup nella LUT, pilotaggio del bus dati. Poiché tutte le funzioni critiche risiedono in IRAM (`IRAM_ATTR`) ed operano solo su registri hardware, il percorso critico è tipicamente inferiore a 20 cicli.

### 4.1 Accesso diretto ai registri GPIO

Per eliminare qualsiasi overhead di astrazione, il firmware usa esclusivamente accesso diretto ai registri hardware dell'ESP32:

- `GPIO.in` — legge i 32 GPIO del bank-0 in un'unica istruzione a 32 bit.
- `GPIO.in1.val` — legge i GPIO del bank-1 (GPIO32–48).
- `GPIO.out_w1ts` / `GPIO.out_w1tc` — set/clear atomico dei bit di uscita (write-1-to-set / write-1-to-clear).
- `GPIO.enable_w1ts` / `GPIO.enable_w1tc` — abilitazione/disabilitazione in uscita (tristate) per il bus dati.

### 4.2 Diagramma Temporale di un Ciclo di Lettura (PBI)

Il diagramma seguente illustra un tipico ciclo di lettura gestito dall'ESP32 in modalità PBI.



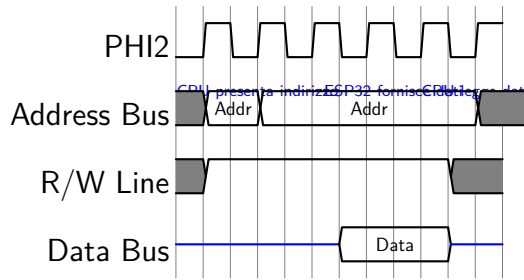


Figura 1: Diagramma temporale di un ciclo di lettura del 6502 gestito dall'ESP32 (modalità PBI).

## 5 Frammento di Codice Sorgente

Di seguito è riportato un estratto aggiornato del file `main.cpp` che mostra le definizioni dei pin e il cuore del `MonitorTask`. Tutte le funzioni critiche sono marcate `IRAM_ATTR` per garantire l'esecuzione dalla RAM interna. L'indirizzo completo A0–A15 è decodificato interamente in software: nessuna logica esterna 74HCT/74LVC.

```

1  /* Bank 0 (GPIO.in) */
2  #define PIN_PHI2          1      /* GPIO1, pin 6 -- Dedicated GPIO input ch
   0 */
3  #define PIN_RW            2      /* GPIO2, pin 7 -- R/W_, HIGH = read
   */
4  /* D0-D7: GPIO 4-11, bank-0 bit 4-11 */
5  #define DBUS_MASK         0x00000FF0UL
6  /* A0-A9: GPIO 12-21, bank-0 bit 12-21 */
7  #define ABUS_LO_MASK      0x003FF000UL
8
9  /* Bank 1 (GPIO.in1.val) */
10 #define PIN_A10           35     /* GPIO35, pin 40 -- bank-1 bit 3 */
11 #define PIN_A11           36     /* GPIO36, pin 41 -- bank-1 bit 4 */
12 #define PIN_A12           33     /* GPIO33, pin 38 -- bank-1 bit 1 */
13 #define PIN_A13           34     /* GPIO34, pin 39 -- bank-1 bit 2 */
14 #define PIN_A14           47     /* GPIO47, pin 53 -- bank-1 bit 15 */
15 #define PIN_A15           48     /* GPIO48, pin 54 -- bank-1 bit 16 */
16 #define PIN_ARESET        37     /* GPIO37, pin 42 -- system reset */
17 #define PIN_CRESET        38     /* GPIO38, pin 43 -- VERA reset */
18 #define PIN_CDONE         39     /* GPIO39, pin 44 -- VERA done */
19 #define PIN_DEV_SEL_N     40     /* GPIO40, pin 45 -- Dedicated out ch 0 */
20 #define PIN_EXTSEL_N      41     /* GPIO41, pin 46 -- Dedicated out ch 1 */
21 #define PIN_MPD           42     /* GPIO42, pin 48 -- Dedicated out ch 2 */
22 #define PIN_RAMBO_EN      3      /* GPIO3, pin 8 -- RAMbo enable */
23
24 /* Dedicated GPIO channel masks */
25 #define DEDIC_IN_PHI2      (1u << 0)
26 #define DEDIC_OUT_DEVSELN (1u << 0)
27 #define DEDIC_OUT_EXTSEL  (1u << 1)
28 #define DEDIC_OUT_MPD      (1u << 2)
29 #define DEDIC_OUT_CTRL_MASK 0x07u
30 #define DEDIC_OUT_CTRL_IDLE 0x07u /* tutti HIGH = deasserted */
31
32 static inline uint16_t IRAM_ATTR decode_addr(uint32_t lo, uint32_t hi)
33 {

```

```

34 uint16_t a = (uint16_t)((lo >> 12) & 0x3FFu);          /* A0-A9 */
35 a |= (uint16_t)(((hi >> (PIN_A10-32)) & 1u) << 10);    /* A10 */
36 a |= (uint16_t)(((hi >> (PIN_A11-32)) & 1u) << 11);    /* A11 */
37 a |= (uint16_t)(((hi >> (PIN_A12-32)) & 1u) << 12);    /* A12 */
38 a |= (uint16_t)(((hi >> (PIN_A13-32)) & 1u) << 13);    /* A13 */
39 a |= (uint16_t)(((hi >> (PIN_A14-32)) & 1u) << 14);    /* A14 */
40 a |= (uint16_t)(((hi >> (PIN_A15-32)) & 1u) << 15);    /* A15 */
41 return a;
42 }

```

Listing 1: Definizioni pin ESP32-S3FN8 (QFN56) – firmware corrente

```

1 static void IRAM_ATTR MonitorTask(void *arg)
2 {
3     /* Bundle Dedicated GPIO: input [PHI2], output [DEV_SEL_N, EXTSEL_N,
4     MPD] */
5     /* ... (setup bundle omissso per brevita') ... */
6     cpu_ll_write_dedic_gpio_mask(DEDIC_OUT_CTRL_MASK,
7     DEDIC_OUT_CTRL_IDLE);
8
9     bool selected = false;
10    bus_release();
11
12    for (;;) {
13        /* 1. Attesa fronte di salita PHI2 via TIE (1 ciclo CPU) */
14        while (!(cpu_ll_read_dedic_gpio_in() & DEDIC_IN_PHI2));
15        uint32_t g_lo = GPIO.in;
16        uint32_t g_hi = GPIO.in1.val;
17
18        /* 2. Decodifica indirizzo A0-A15 (16 bit, software puro) */
19        bool is_read = (g_lo & (1UL << PIN_RW)) != 0;
20        uint16_t addr = decode_addr(g_lo, g_hi);
21        uint8_t off8 = (uint8_t)(addr & 0xFFu);
22
23        bool is_d1xx = (addr >= 0xD100u && addr <= 0xD1FFu);
24        bool is_d6xx = (addr >= 0xD600u && addr <= 0xD7FFu);
25        bool is_d8xx = (addr >= 0xD800u && addr <= 0xDFFFu);
26        bool is_vera_range = is_d1xx && (off8 <= 0x1Fu);
27        bool is_vcs_latch = is_d1xx && (off8 == 0xFFu);
28        bool is_rambo_win = (addr >= 0x4000u && addr <= 0x7FFFu);
29        bool rambo_active = rambo_hw_enabled && ((PORTB & 0x10u) == 0u)
30    ;
31
32        /* 3. Asserzione segnali controllo (1 istruzione TIE) */
33        {
34            uint8_t ctrl = DEDIC_OUT_CTRL_IDLE;
35            if (selected) {
36                if ((is_d1xx && !is_vcs_latch) || is_d6xx)
37                    ctrl &= ~DEDIC_OUT_EXTSEL;
38                if (is_d8xx) ctrl &= ~DEDIC_OUT_MPD;
39                if (is_vera_range) ctrl &= ~DEDIC_OUT_DEVSELN;
40            }
41            if (rambo_active && is_rambo_win)
42                ctrl &= ~DEDIC_OUT_EXTSEL;
43            cpu_ll_write_dedic_gpio_mask(DEDIC_OUT_CTRL_MASK, ctrl);
44        }
45    }
46 }

```

```

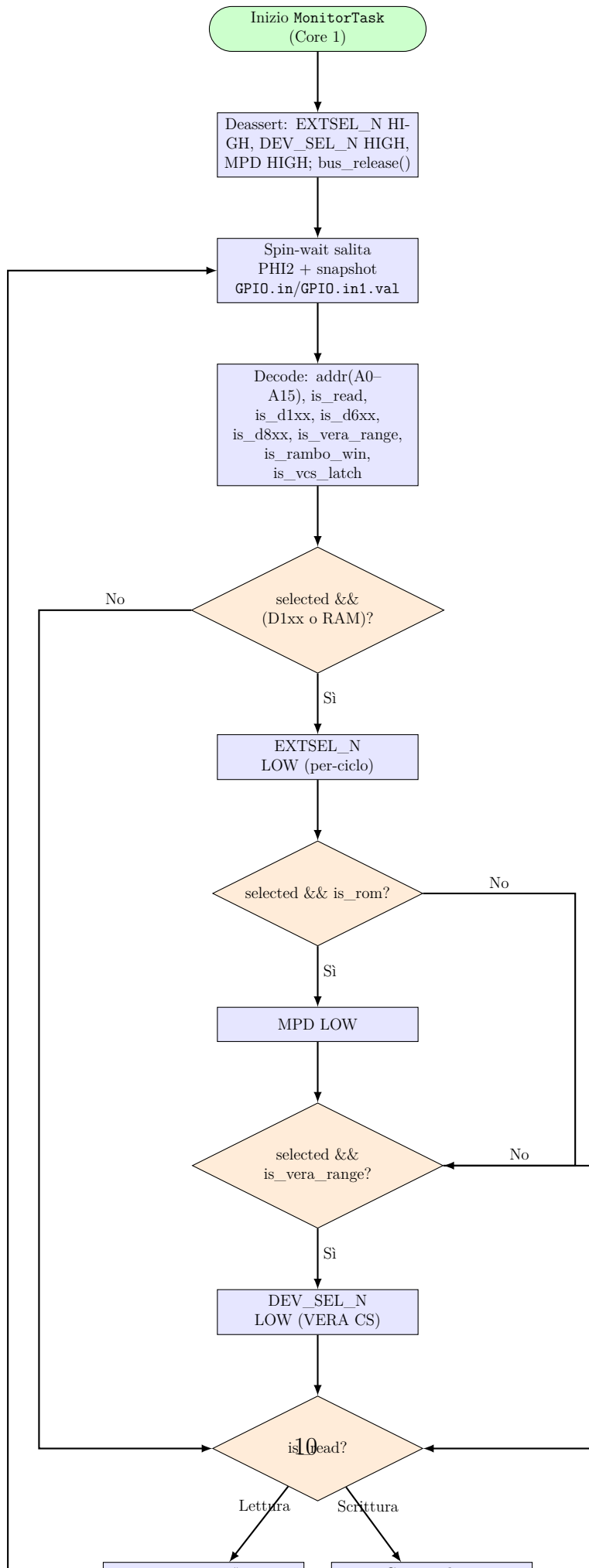
42
43     /* 4. Ciclo di lettura */
44     if (is_read) {
45         if (selected && is_d8xx)
46             bus_drive(pbi_driver[addr & 0x7FFu]);
47         else if (selected && is_d6xx)
48             bus_drive(ram_pbi[addr & 0x1FFu]);
49         if (rambo_active && is_rambo_win) {
50             uint8_t bank = ((PORTB>>2)&3u)|((PORTB>>3)&0xCu);
51             bus_drive(extended_rambo_256k[((uint32_t)bank<<14)|(addr
&0x3FFFu)]));
52         }
53     }
54     /* 5. Ciclo di scrittura */
55     else {
56         uint8_t data = decode_data(GPIO.in);
57         if (is_vcs_latch) {
58             selected = (data == PBI_DEV_ID);
59         }
60         /* Snoop PIA 6520 (passivo -- il 6520 fisico risponde; ESP32
solo osserva) */
61         if (addr == 0xD303u) PBCTL = data;
62         if (addr == 0xD301u && (PBCTL & 0x04u)) PORTB = data;
63         if (rambo_active && is_rambo_win) {
64             uint8_t bank = ((PORTB>>2)&3u)|((PORTB>>3)&0xCu);
65             extended_rambo_256k[((uint32_t)bank<<14)|(addr&0x3FFFu)]
= data;
66         }
67     }
68
69     /* 6. Fine ciclo: rilascia bus, deasserta controlli */
70     while (cpu_ll_read_dedic_gpio_in() & DEDIC_IN_PHI2);
71     bus_release();
72     cpu_ll_write_dedic_gpio_mask(DEDIC_OUT_CTRL_MASK,
DEDIC_OUT_CTRL_IDLE);
73 }
74 }

```

Listing 2: Nucleo del MonitorTask – hot loop semplificato (PBI + RAMbo)

## 6 Diagramma di Flusso del Firmware

La logica principale del firmware è contenuta nel `MonitorTask`, che viene eseguito sul secondo core dell'ESP32 (Core 1) con priorità massima per garantire prestazioni in tempo reale.



## 7 Conclusione

L'analisi conferma che l'architettura hardware e software del progetto è solida e performante. L'ESP32-S3FN8 a 240 MHz garantisce circa 67 cicli di clock nella finestra critica di PHI2 alto (279 ns), ampiamente sufficienti per l'intera catena di decodifica e pilotaggio del bus. Le tecniche di programmazione a basso livello — accesso diretto ai registri GPIO (GPIO.in, GPIO.in1.val, GPIO.out\_w1ts/w1tc, GPIO.out1\_w1tc/out1\_w1ts), funzioni in IRAM, LUT pre-calcolata per il bus dati — garantiscono un percorso critico tipicamente inferiore a 20 cicli.

### Segnali di controllo revisionati

La revisione del firmware e della mappatura hardware ha chiarito e corretto i seguenti aspetti:

- **EXTSEL\_N** (GPIO 41, Dedicated GPIO output ch 1) è un segnale *per-ciclo*: viene asserito basso quando il device è selezionato (**selected=true**) e l'accesso ricade in \$D100–\$D1FE (escluso \$D1FF) oppure in \$D600–\$D7FF; oppure quando RAMbo è attivo e l'accesso è nella finestra \$4000–\$7FFF (indipendentemente da **selected**). Viene sempre deassertato al termine del ciclo. Non è un segnale persistente legato allo stato del latch VCS.
- **MPD (Math Pack Disable)** (GPIO 42, Dedicated GPIO output ch 2) viene asserito basso esclusivamente quando il device è selezionato e l'accesso riguarda la ROM interna \$D800–\$DFFF. Non viene asserito per accessi alla RAM, a VERA o alla finestra RAMbo. Viene asserito insieme a EXTSEL\_N e DEV\_SEL\_N in un'unica istruzione TIE (`cpu_ll_write_dedic_gpio_mask`).
- **RAM interna PBI 512 B** (\$D600–\$D7FF): indirizzata su 9 bit via `ram_pbi[addr & 0x1FFu]`. Rilevata per confronto diretto dell'indirizzo a 16 bit (`addr >= 0xD600u && addr <= 0xD7FFu`): non esiste più il segnale hardware RAM\_SEL\_N. GPIO 40 (MTDO, QFN56 pin 45) è ora DEV\_SEL\_N (VERA chip select).
- **Decodifica indirizzi software (nessuna logica 74HCT/74LVC)**: il firmware precedente utilizzava IC 74HCT e 74LVC per pre-decodificare gli indirizzi e generare segnali (D1XX\_N, ROM\_SEL\_N, RAM\_SEL\_N) come ingressi ESP32. Nel firmware attuale questi segnali sono stati eliminati: l'ESP32 legge A0–A15 direttamente (tramite i quattro TXS0108E U1–U4, usati solo per adattamento di livello 5 V ↔ 3.3 V) e decodifica tutto in software tramite `decode_addr()`.
- **Ruolo attuale di U3**: il TXS0108E U3 trasporta A8, A9, A10, A11 (ingressi B→A), PHI2, R/W\_ (ingressi B→A), EXTSEL\_N e MPD (uscite A→B) e ARESET (uscita A→B). Tutti gli 8 canali di U3 sono occupati; nessun canale è disponibile. La quarta istanza di TXS0108E (U4) è dedicata esclusivamente a A12–A15.
- **Snooping passivo della PIA 6520 — PBCTL (\$D303) e PORTB (\$D301)**: il chip **6520 PIA** è fisicamente presente sulla scheda madre dell'Atari e risponde

a \$D301/\$D303 normalmente. Il firmware **non emula** il 6520: lo **snoopa** in modo passivo. La distinzione operativa è:

| Indirizzo       | Chi risponde              | Ruolo ESP32                          |
|-----------------|---------------------------|--------------------------------------|
| \$D301 / \$D303 | 6520 PIA (hardware reale) | Solo snooping — non pilota mai D0–D7 |
| \$4000–\$7FFF   | ESP32 (RAMbo window)      | Attivo: EXTSEL_N + pilota D0–D7      |

Quando il 6502 scrive a \$D303 o \$D301, il 6520 riceve e processa la transazione; l'ESP32 osserva gli stessi dati sul bus e aggiorna le shadow copy locali (PBCTL, PORTB). Non vi è mai contesa sul bus perché l'ESP32 non guida il bus dati per quegli indirizzi.

La PIA espone due registri sovrapposti a \$D301: il *Data Direction Register* (DDR) quando PBCTL bit 2 = 0, e l'*Output Register* (PORTB, bank selection) quando PBCTL bit 2 = 1. Il firmware snoopa \$D303 ad ogni write cycle e aggiorna PBCTL; le write a \$D301 aggiornano PORTB **solo** quando  $\text{PBCTL} \& 0x04 \neq 0$ .

Il tracciamento di PBCTL è necessario *precisamente perché* lo snooping è passivo: dal bus, una write DDR e una write all'Output Register su \$D301 sono **indistinguibili** (stesse linee, stesso ciclo). Senza lo stato di PBCTL, le write DDR della sequenza di init del sistema operativo ( $0xFF \rightarrow \$D301$  con  $\text{PBCTL} = 0$ ) verrebbero erroneamente interpretate come selezioni di banco. Rif.: datasheet Motorola 6520 PIA; Altirra Hardware Reference Manual §8; forum AtariAge (utente *warerat*).

- **GPIO 15 e GPIO 16** (pad RTC XTAL\_32K\_P/N): il progetto non impiega quarzo da 32 kHz; i pad sono utilizzati come normali GPIO per segnali indirizzo. Il MCU richiede invece un quarzo esterno a **40 MHz** sulle pad dedicate **XTAL\_P** (pin 56) e **XTAL\_N** (pin 55) — senza di esso il chip non si avvia (vedere sezione 3.1).

Il firmware supporta le due modalità ( $\text{VERA\_BOARD\_IS\_PBI}=1$  PBI,  $\text{VERA\_BOARD\_IS\_PBI}=0$  CCTL) in un unico ambiente di build (`esp32s3fn8`), senza duplicazione di codice. La tabella dei pin fisici del package QFN56 consente di realizzare il PCB direttamente dalla documentazione senza ambiguità tra numero GPIO logico e pad fisico.